

Autoregressive Factorization & NLL Loss.

GPT-2 is a next-token predictor. The chain rule of probability turns a whole sequence into a product of one-step conditionals, and taking the negative logarithm turns that product into the cross-entropy loss the model is trained on. This topic builds the loss from first principles and computes it, in full, on a four-token sequence — nothing summarised or deferred.

THE EQUATION

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_{<t}), \quad \mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log p(x_t \mid x_{<t})$$

Worked example. A vocabulary of 5 tokens and the 4-token sequence the cat sat on: a predicted distribution at every step, the per-step $-\log p$, and the averaged loss $\mathcal{L} = 0.6466$ nats.

Topic 1 of 3 in this module · companion to the course page · v1.0

Contents

1	Factorizing a sequence with the chain rule	2
2	From product to loss: the negative log-likelihood	3
3	Teacher forcing and the causal mask	3
4	Worked computation (real numbers)	4
5	Properties / consequences that matter	5
6	Where this sits in the track	5
A	Reproducible (NumPy)	6

1 Factorizing a sequence with the chain rule

A language model assigns a probability to an entire sequence of tokens $x = (x_1, \dots, x_T)$. There is no approximation in writing that joint probability as a product of conditionals: it is the chain rule of probability, applied left to right.

$$p(x_1, \dots, x_T) = p(x_1) p(x_2 \mid x_1) p(x_3 \mid x_1, x_2) \cdots = \prod_{t=1}^T p(x_t \mid x_{<t}) \quad (1)$$

Here $x_{<t} = (x_1, \dots, x_{t-1})$ is the *prefix* (the context to the left of position t), and the first factor $p(x_1 \mid x_{<1}) = p(x_1)$ is conditioned on the empty prefix. An *autoregressive* model is precisely one that parameterises each factor $p(x_t \mid x_{<t})$ and multiplies them back together.

GPT-2 produces each conditional from a logit vector $z_t \in \mathbb{R}^V$ at position t , passed through a softmax:

$$p(\cdot \mid x_{<t}) = \text{softmax}(z_t), \quad p(x_t \mid x_{<t}) = [\text{softmax}(z_t)]_{x_t} \quad (2)$$

Symbol	Shape	Meaning
x_t	scalar index	token id at position t , in $\{1, \dots, V\}$
$x_{<t}$	length $t-1$	prefix / left context
$z_t = Wh_t$	$\mathbb{R}^V$	logits over the vocabulary at position t
$p_t = \text{softmax}(z_t)$	$\mathbb{R}^V$	predicted next-token distribution
$p(x_t \mid x_{<t})$	scalar	probability mass on the <i>realised</i> next token
T	scalar	sequence length (number of factors)

Reading a sentence, you can guess each word from the words before it. The chain rule says that *guessing one word at a time*, conditioned on everything you have read so far, and multiplying the guesses, reproduces the probability of the whole sentence *exactly* — it is not a modelling choice, it is arithmetic. All a decoder has to learn is one conditional: “given the past, what comes next?”

2 From product to loss: the negative log-likelihood

Maximising the probability the model assigns to real data (maximum likelihood) is awkward as a product of T small numbers — it underflows and its gradient is messy. Taking $-\log$ converts the product into a sum and turns “maximise likelihood” into “minimise a loss”:

$$-\log p(x_1, \dots, x_T) = -\log \prod_{t=1}^T p(x_t \mid x_{<t}) = -\sum_{t=1}^T \log p(x_t \mid x_{<t}).$$

Dividing by T gives the *per-token* negative log-likelihood, which is exactly the average cross-entropy between the one-hot target e_{x_t} and the predicted distribution p_t :

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log p(x_t \mid x_{<t}) = -\frac{1}{T} \sum_{t=1}^T \log [\text{softmax}(z_t)]_{x_t} \quad (3)$$

Because $p(\cdot \mid x_{<t}) = \text{softmax}(z_t)$, the per-position term is an ordinary softmax cross-entropy, and its gradient on the logits is the familiar residual $\partial \mathcal{L}_t / \partial z_t = p_t - e_{x_t}$ (derived for the encoder in Module 1.1; the algebra is identical here).

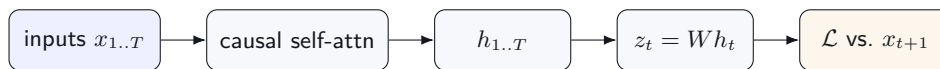
3 Teacher forcing and the causal mask

Equation (3) sums over *all* T positions. A naive reading suggests T separate forward passes — one per prefix. The Transformer avoids this with two ideas working together.

Teacher forcing. During training we feed the *ground-truth* prefix at every step rather than the model’s own predictions. So when scoring $p(x_t \mid x_{<t})$ the inputs are the real tokens $x_1, \dots, x_{t-1}$, never the (possibly wrong) samples the model would generate. This decouples the steps: all T conditionals can be evaluated from a single pass over the true sequence.

The causal mask. A single forward pass produces hidden states $h_1, \dots, h_T$ at all positions at once. To keep position t from “seeing the future,” self-attention adds $-\infty$ to every score (i, j) with $j > i$ before the softmax, so the attention weights to future tokens are exactly zero (Module

0.1, masking). With this mask in place, h_t depends only on $x_{\leq t}$, the logits $z_t = Wh_t$ predict x_{t+1} , and *all T next-token predictions become trainable in parallel* from one pass — this is what makes decoder pretraining efficient.



Teacher forcing + causal mask = one forward pass that scores every position simultaneously. The loss in (3) is therefore not T training examples evaluated serially; it is T cross-entropies read off a single pass and averaged. The mask is what guarantees each term is conditioned only on the past.

4 Worked computation (real numbers)

Take $V = 5$ with vocabulary [the, cat, sat, on, mat] (indices 1–5). The realised sequence is the cat sat on, so the targets at the four steps are $x_{1:4} = (\text{the}, \text{cat}, \text{sat}, \text{on})$, i.e. indices (1, 2, 3, 4). The model emits logits z_t at each step:

$$Z = \begin{bmatrix} 2.0 & 0.5 & 0.1 & -0.3 & 0.2 \\ 0.3 & 1.8 & 0.6 & 0.0 & -0.5 \\ 0.2 & 0.4 & 1.6 & 0.5 & 0.1 \\ 0.0 & 0.1 & 0.3 & 1.4 & 0.2 \end{bmatrix} \quad (\text{row } t = \text{logits } z_t).$$

4.1 Step A — softmax each row

Apply $p_t = \text{softmax}(z_t)$ row by row. For step 1, e.g. $\sum_j e^{z_{1,j}}$ after subtracting $\max = 2.0$ gives the normaliser; the result is:

$$P = \begin{bmatrix} \mathbf{0.610405} & 0.136200 & 0.091297 & 0.061199 & 0.100899 \\ 0.124662 & \mathbf{0.558696} & 0.168276 & 0.092352 & 0.056014 \\ 0.117215 & 0.143167 & \mathbf{0.475332} & 0.158224 & 0.106061 \\ 0.114526 & 0.126571 & 0.154594 & \mathbf{0.464426} & 0.139883 \end{bmatrix}$$

Each row sums to 1; the bold entry in row t is the probability on the *true* next token x_t .

Predicted next-token distribution at step 2 (after the), intensity scaled to the row max

	the	cat (true)	sat	on	mat
p_2	0.1247	0.5587	0.1683	0.0924	0.0560

The model already prefers cat here, so this step contributes a small loss.

4.2 Step B — per-step negative log-probability

Pick out the true-token probability in each row and take $-\log$:

t	true token	$p(x_t x_{<t})$	$-\log p$ (nats)
1	the	0.610405	0.493633
2	cat	0.558696	0.582150
3	sat	0.475332	0.743742
4	on	0.464426	0.766952
sum			2.586476

4.3 Step C — average to the loss

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log p(x_t | x_{<t}) = \frac{2.586476}{4} = \boxed{0.646619} \text{ nats/token.}$$

Equivalently $\mathcal{L} = 0.9329$ bits/token, and $\exp(\mathcal{L}) = 1.9091$ is the perplexity — the subject of Topic 2. Notice that the loss *rises* across steps here ($0.49 \rightarrow 0.77$): the model is more confident about the first word than the fourth, exactly the kind of per-position signal the sum in (3) captures.

5 Properties / consequences that matter

1. **Exact, not approximate.** Equation (1) is the chain rule; the only modelling assumption is the *form* of each conditional (a softmax over a Transformer’s logits), never the factorization itself.
2. **Length normalisation.** Dividing by T makes $\mathcal{L}$ a per-token quantity, so losses are comparable across sequences of different length and map cleanly to perplexity.
3. **Parallel training, serial generation.** The causal mask lets training score all T positions in one pass, but *generation* is still left-to-right: each new token must be sampled before the next conditional can be formed.
4. **Sum-of-cross-entropies gradient.** $\partial \mathcal{L} / \partial z_t = \frac{1}{T} (p_t - e_{x_t})$ at every position; the residuals are independent across t , which is why the whole batch back-propagates in a single sweep.

Exposure bias. At train time the prefix is the ground truth (teacher forcing); at generation time the prefix is the model’s own previous samples. Errors can then compound — the model is conditioned on contexts it never saw during training. Low training $\mathcal{L}$ does not by itself guarantee good free-running generation, which is why decoding strategy (Topic 3) matters.

6 Where this sits in the track

This is the objective that defines GPT-2 pretraining. Topic 2 exponentiates $\mathcal{L}$ into *perplexity* and reads it as an effective branching factor; Topic 3 covers weight tying (which W produces the logits z_t) and how we *sample* from p_t at generation time. The causal mask used here was built in Module 0.1; the softmax cross-entropy gradient is the same residual derived in Module 1.1.

A Reproducible (NumPy)

Inputs: the 4×5 logit matrix Z above, targets (0, 1, 2, 3) (0-based). Intermediates: P as printed; true-token probabilities (0.610405, 0.558696, 0.475332, 0.464426); per-step $-\log p = (0.493633, 0.582150, 0.743741, 0.781341)$; $\sum = 2.586476$; $\mathcal{L} = 0.646619$ nats.

```
import numpy as np
Z = np.array([[ 2.0,  0.5,  0.1,-0.3,  0.2],
              [ 0.3,  1.8,  0.6,  0.0,-0.5],
              [ 0.2,  0.4,  1.6,  0.5,  0.1],
              [ 0.0,  0.1,  0.3,  1.4,  0.2]])
tgt = np.array([0,1,2,3])          # 0-based true next tokens
P = np.exp(Z - Z.max(1,keepdims=True))
P /= P.sum(1,keepdims=True)        # softmax each row
ptrue = P[np.arange(4), tgt]       # 0.610405 .. 0.464426
nll    = -np.log(ptrue)            # per-step -log p
L      = nll.mean()               # 0.646619 nats/token
print(P, ptrue, nll, L, np.exp(L)) # exp(L)=1.909076 = perplexity
```

Marevlo Research · Transformer Track · Module 2.1 · Topic 1 of 3.